



UNIVERSIDAD
DE GRANADA

Departamento de Ciencias de la
Computación e Inteligencia Artificial

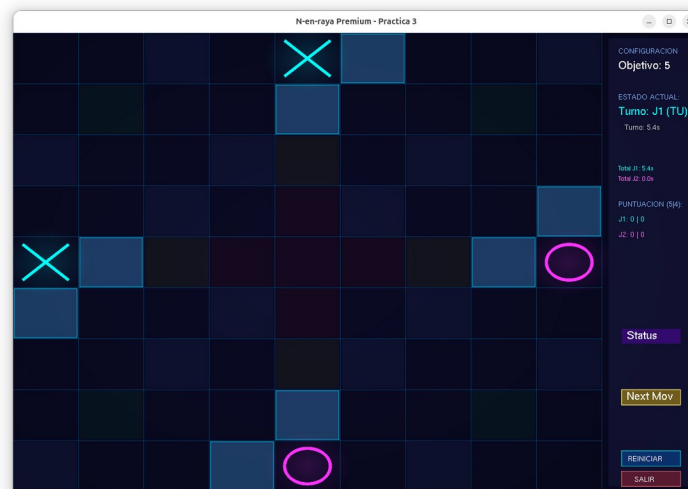
/ UGR / decsai

INTELIGENCIA ARTIFICIAL

E.T.S. de Ingenierías Informática y de
Telecomunicación

Práctica 3

5-en-Raya Táctico (Modo Competición 9x9)



Búsqueda con Adversario (Juegos)



DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN E INTELIGENCIA
ARTIFICIAL
UNIVERSIDAD DE GRANADA
Curso 2025-2026

1. Introducción

La tercera práctica de la asignatura consiste en el diseño e implementación de técnicas de búsqueda con adversario en un entorno de juegos. En esta ocasión, trabajaremos con una versión avanzada y altamente estratégica del clásico juego de las "Tres en Raya", denominada **5-en-Raya Táctico (Modo Competición 9x9)**.

A diferencia del juego tradicional, esta variante introduce restricciones de colocación, turnos asimétricos y casillas con efectos especiales que amplían enormemente el espacio de estados y el desafío computacional.

El objetivo de la práctica es que el estudiante muestre tener la capacidad de implementar los algoritmos **Status**, **Minimax** y **Poda Alfa-Beta** sobre un juego concreto, y que además es capaz de dotar de un comportamiento inteligente deliberativo a un agente para competir contra oponentes en ese juego.

2. Requisitos

Para poder realizar la práctica, es necesario que el/la alumno/a disponga de:

1. Conocimientos básicos del lenguaje C/C++: tipos de datos, sentencias condicionales, sentencias repetitivas, funciones y procedimientos, clases, métodos de clases, constructores de clase.
2. El software para construir el simulador `n_en_raya` está disponible en el GitHub de la asignatura.



3. Objetivo de la práctica

En esta última práctica de la asignatura jugaremos con esos juegos clásicos consistentes en intentar alinear fichas del mismo jugador sobre un tablero cuadrulado. Junto a esta práctica se provee de un software que generaliza al conocido juego 3 en raya, para poder jugar sobre tableros de tamaño más grande y decidir el número de fichas que deben alinearse para ganar.

El estudiante deberá implementar la lógica de decisión de un agente inteligente en el archivo *AgenteEstudiante.cpp*. Los objetivos técnicos incluyen:

- Implementación del algoritmo **Status**
- Implementación del algoritmo **MiniMax**
- Implementación de la **Poda Alfa-Beta**.
- Diseño de una **función heurística** que defina un buen jugador del juego del 5-en-Raya en tablero de 9x9.

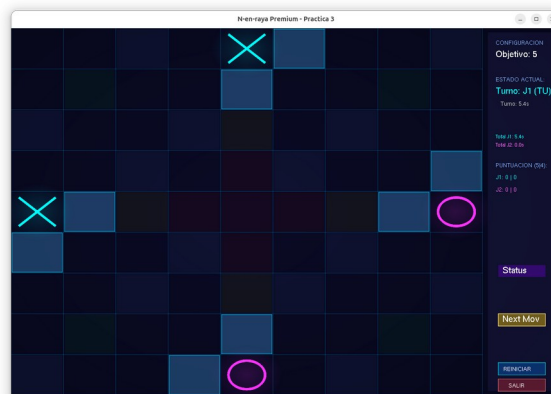
En la **versión de competición** de la práctica trabajaremos sobre un **tablero 9x9** siendo el objetivo conseguir **alinear 5 fichas** del mismo jugador de forma horizontal, vertical o diagonal.

Para el resto de configuraciones distintas a la de competición, regirán las reglas clásicas. Sin embargo, para la configuración de competición y exclusivamente para esta configuración, se define el siguiente conjunto de reglas especiales:

REGLAS MODO COMPETICIÓN

1. **Objetivo del Juego:** Participan dos jugadores (J1 (X) y J2 (O)). El objetivo es ser el primero en conseguir alinear **5 piezas consecutivas** de su propio color en línea recta (horizontal, vertical o diagonal).

2. **Tablero e Inicio:** El tablero estándar es de **9x9** casillas. Para garantizar la conectividad desde el primer instante, la partida no empieza vacía: se inicia con **4 piezas pre-colocadas** en el centro (2 del J1 y 2 del J2).



3. **Secuencia de Turnos (1-2-2-2):** El flujo de colocación es asimétrico. En el Turno 1, el J1 coloca únicamente **1 pieza**. En todos los turnos sucesivos, los jugadores se alternan colocando un máximo de **2 piezas consecutivas** por turno.
4. **La Regla de la Trinidad:** El espacio de juego se divide en tres fases matemáticas. Un jugador solo puede colocar una pieza en una casilla (fila, columna) si el sumatorio de sus coordenadas cumple: $(\text{fila} + \text{columna}) \% 3 == \text{faseActual} \% 3$. La faseActual empieza en 2 y avanza +1 cuando el jugador agota sus movimientos y cede el turno.
5. **La Regla de Adyacencia:** Exceptuando la configuración inicial, toda pieza que se coloque en el tablero debe situarse obligatoriamente en una casilla vacía que sea **adyacente** (en cualquiera de las 8 direcciones) a una pieza ya existente.
6. **Casillas Verdes (Místicas):** Cuando una pieza aterriza en una casilla verde, el jugador gana de forma inmediata **+1 movimiento extra** para usar en su turno actual.
7. **Casillas Rojas (Sabotaje):** Estas casillas actúan como trampas. Si un jugador sitúa su pieza aquí, el tablero la absorberá y la transformará automáticamente en una pieza del **color del oponente**.

8. ● **Casillas Amarillas (Bomba):** Al colocar una pieza en esta celda, se produce una detonación que **elimina** (vuelve a dejar vacías) todas las celdas de la misma fila y la misma columna, respetando únicamente la posición de la bomba.
9. **Resolución por Empate (Tie-break):** Si el tablero se satura por completo sin que nadie logre el objetivo, el sistema otorgará la victoria a quién posea más líneas de 4 piezas. Si también tienen las mismas, la partida se dará por empatada.

4. Instalación y descripción del simulador

4.1 Instalación del simulador

El simulador *n_en_raya* nos permitirá

- Implementar el comportamiento de uno o dos jugadores en un entorno en el que el jugador (bien humano o bien máquina) podrá competir con otro jugador software o con otro humano.
- Visualizar los movimientos decididos en una interfaz de usuario.

Para instalarlo, debe seguir los pasos especificados en el GitHub <https://github.com/ugr-ccia-IA/practica3> de la asignatura.

4.2 Parámetros del simulador en línea de comandos

La primera vez que se descarga el software, la forma más simple de instalación es invocando al *./install.sh*. Este procedimiento instalará los paquetes necesarios para poder compilar el simulador y hará la primera compilación. Para el resto de las veces, la compilación se podrá hacer con un *make -j*.

Una vez compilado el software nos aparecerán 2 ficheros ejecutables *./n_en_raya* y *./n_en_rayaSH*. El primero de ellos usa hebras para no dejar bloqueada la interfaz gráfica

Departamento de Ciencias de la Computación e Inteligencia Artificial

pero puede dar problemas en algunas máquinas. En esas otras máquinas se puede usar la versión con SH (Sin Hebras) que es una versión más compatible.

El binario principal `./n_en_raya` permite una configuración total de la partida mediante parámetros. Esto es especialmente útil para realizar experimentos de rendimiento, comparar heurísticas o ejecutar pruebas automáticas sin interfaz gráfica.

Definición de Jugadores (-p1 y -p2)

Define el tipo de agente que controlará a cada jugador.

- * humano (por defecto): El jugador es controlado por una persona a través de la interfaz gráfica o la consola.
- * aleatorio: El agente elige cualquier casilla válida de forma totalmente al azar. Útil como oponente básico para pruebas iniciales.
- * minimax: Activa el agente con búsqueda MiniMax (implementado por el estudiante).
- * inteligente: Activa el agente con Poda Alfa-Beta (implementado por el estudiante).
- * status: Activa la búsqueda exhaustiva completa para encontrar el veredicto teórico (Gana/Pierde/Empata). **Atención:** Ignora el límite de tiempo (-t) y solo es viable en tableros pequeños.
- * ninja1, ninja2, ninja3, ninja4: Agentes pre-entrenados con distintos niveles de dificultad que actúan como “rivales” a batir.

Configuración del Tablero (-f, -c, -n)

Permite cambiar las dimensiones y las reglas de victoria.

- f <int>: Número de filas. (Defecto: 9).
- c <int>: Número de columnas. (Defecto: 9).
- n <int>: Número de piezas consecutivas necesarias para ganar. (Defecto: 5).

[!**IMPORTANTE**] Cambiar estas dimensiones afecta directamente al juego. Solo en el **Modo Competición (9x9 con n=5)** se aplican las reglas de Turnos, Trinidad y Adyacencia descritas anteriormente. Para el resto de configuraciones, los turnos son alternos de un movimiento, no hay ni regla de Adyacencia, ni de Trinidad.

Parámetros de la IA (-d, -t, -id1, -id2)

Controlan el comportamiento del algoritmo Minimax.

Departamento de Ciencias de la Computación e Inteligencia Artificial

-d <int>: Profundidad máxima de búsqueda. Un valor más alto hace a la IA más “sabia” pero consume mucho más tiempo. (Defecto: 4).

-t <double>: Límite de tiempo en segundos por movimiento. La IA utiliza este valor para abortar su búsqueda si se está excediendo. (Defecto: 100.0).

-id1 <int> / -id2 <int>: Selector de heurística. Permite elegir qué función de evaluación usar para el J1 o J2 respectivamente.

Otros Parámetros

-h: Muestra el mensaje de ayuda con un resumen de todas las opciones.

-nogui: Desactiva la interfaz gráfica. La partida se ejecuta a máxima velocidad y el tablero se imprime por consola en cada turno. Imprescindible para realizar experimentos masivos (benchmarking).

-file <nombreFichero>: Carga un estado de tablero desde un fichero de texto. Esto anula los parámetros -f, -c y -n. Esta opción permite meter nuevos tableros donde aparezcan situaciones intermedias del juego para analizar o estudiar el comportamiento de los algoritmos utilizados en el desarrollo de la práctica. A continuación se indica el formato que debe tener ese fichero.

Formato del Fichero de Tablero (-file)

El fichero debe ser un texto plano con la siguiente estructura:

<pre>filas columnas objetivo fase jugadorTurno movs_rest turno_actual [rejilla de 0, 1, 2]</pre>
--

- **fase**: Valor que determina el residuo $(f+c)\%3$ permitido (avanza cuando cambia el jugador).
- **jugadorTurno**: 1 o 2.
- **movs_rest**: Número de piezas que le quedan por colocar al jugador en este turno (secuencia 1-2-2-2).
- **turno_actual**: Número total de piezas que ya han sido colocadas en el tablero (plies).
- **rejilla**: Matriz de enteros separados por espacios (0: vacío, 1: J1, 2: J2).



Ejemplos Avanzados de Ejecución

Ejemplo A: Entrenamiento contra Ninja de nivel 2

```
./n_en_raya -p1 humano -p2 ninja2 -f 9 -c 9 -n 5
```

Ideal para practicar estrategias contra un oponente de nivel medio antes de programar tu propia IA.

Ejemplo B: Experimento de Profundidad (Modo Texto)

```
./n_en_raya -p1 inteligente -id1 1 -p2 inteligente -id2 1 -d 3 -t 10.0  
-nogui
```

Enfrenta a la misma IA contra sí misma a profundidad 7. Al no haber interfaz, la partida se resolverá rápidamente y verás el log de nodos visitados en la terminal.

Ejemplo C: Comparativa de Heurísticas

```
./n_en_raya -p1 inteligente -id1 1 -p2 inteligente -id2 2 -d 4 -nogui
```

Duelo entre tu heurística (ID 1) y tu heurística básica (ID 2). Es la mejor forma de validar científicamente si tus mejoras en la función de evaluación son realmente efectivas.

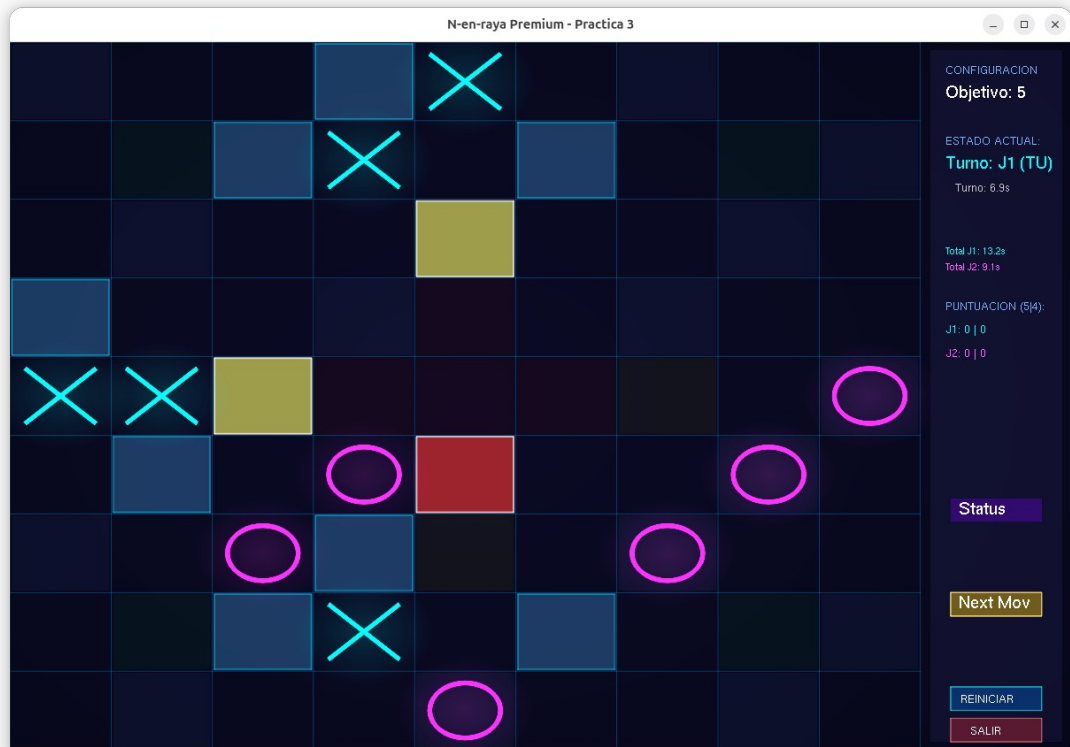
Ejemplo D: Resolución Completa (Tablero 3x3)

```
./n_en_raya -p1 status -f 3 -c 3 -n 3 -nogui
```

Intenta resolver el 3-en-rayas clásico. Verás por consola si el primer movimiento lleva a una victoria forzada o a un empate.

4.3 Parámetros del simulador en modo gráfico

La interfaz gráfica de esta práctica ha sido diseñada utilizando **OpenGL** con una estética *Premium* basada en *Glassmorphism* y efectos de neón. Este documento describe los elementos visuales, el panel de control y las funcionalidades disponibles.



El Tablero de Juego

El tablero es una rejilla de tamaño variable (por defecto 9x9). Las celdas se comportan según la **Regla de la Trinidad** y las casillas especiales.

Elementos Visuales:

- **Fichas Cian (X):** Pertenecen al Jugador 1.
- **Fichas Magenta (O):** Pertenecen al Jugador 2.
- **Efecto Neón:** Las fichas recién colocadas emiten un resplandor del color del jugador.
- **Línea Ganadora:** Cuando un jugador gana, se dibuja una línea gruesa traslúcida uniendo las piezas que forman el “N-en- raya”.

Tipos de Casillas:

- **Negras/Azul oscuro:** Casillas normales.
- **Verdes (Místicas):** Permiten repetir turno.

Departamento de Ciencias de la Computación e Inteligencia Artificial

- **Rojas (Sabotaje):** Colocan una ficha del oponente en lugar de la propia.
- **Amarillas (Bomba):** Limpian la fila y columna al ser activadas.

Panel Lateral de Información

Situado a la derecha, este panel traslúcido muestra el estado en tiempo real de la partida y la IA.

Bloque de Configuración:

- **Objetivo:** Indica cuántas piezas en raya se necesitan para ganar (ej: Objetivo: 5).
- **Turno:** Indica a quién le toca mover (J1 o J2) y si ese jugador es una **IA** o un **Humano (TU)**.

Bloque de Estado Actual:

- **Cronómetro de Turno:** Tiempo que lleva el jugador actual pensando su jugada.
- **Tiempos Acumulados:** Tiempo total consumido por cada jugador a lo largo de toda la partida.
- **Marcador de Puntos (Tiebreaker):** En caso de que el tablero se llene, se muestran las líneas de 5 y 4 conseguidas por cada jugador (ej: J1: 2 | 5 significa 2 líneas de cinco y 5 líneas de cuatro).

Botones de Control

Botón Next Mov (Sugerencia)

Este botón es una herramienta de asistencia y depuración para el jugador humano.

* **Funcionalidad:** Al pulsarlo, el sistema invoca una instancia temporal de la IA del estudiante. Esta IA analizará el tablero con la profundidad configurada (-d) y realizará el mejor movimiento posible automáticamente.

* **Estado “Thinking...”:** Mientras la IA calcula el movimiento, el botón cambiará a color naranja y mostrará el texto “Thinking...”. Durante este proceso, la interfaz permanece activa pero no se permiten nuevos movimientos hasta que la IA termine.

* **Uso Recomendado:** Úsalo para ver qué decisión tomaría tu heurística en una posición comprometida.

Botón Status (Análisis Completo)

Es la herramienta más potente de análisis estratégico del software.

* **Funcionalidad:** Inicia una búsqueda exhaustiva (Full Search) para resolver el juego desde el estado actual. A diferencia del botón Next Mov, este no usa heurísticas, sino que intenta encontrar el resultado teórico perfecto si ambos jugadores jugaran de forma óptima.

* **Feedback Visual:** Una vez calculado, mostrará sobre el botón el veredicto (ej: GANA J1 o EMPATE) y las coordenadas de la casilla que lleva a ese resultado.

* **Limitación:** Debido a la explosión combinatoria, este botón solo es efectivo en tableros pequeños o finales de partida muy avanzados. En tableros de 9x9 vacíos, el tiempo de cálculo puede ser excesivo.

● **Botón REINICIAR**

- Limpia el tablero por completo.
- Resetea los cronómetros de tiempo acumulado.
- Vuelve el turno al Jugador 1.
- *Atajo de teclado:* Tecla R.

● **Botón SALIR**

- Cierra la aplicación de forma segura.
- *Atajo de teclado:* Tecla ESC.

5. Pasos para construir la práctica

Para implementar vuestra solución, el alumno deberá modificar únicamente:

- AgenteEstudiante.cpp y hpp, donde se implementa la clase AgenteEstudiante usada para representar a cada uno de los dos jugadores inteligentes. Estos archivos son los únicos que modificaréis y contendrán TODA LA LÓGICA de vuestra solución. Además, podéis consultar (sin modificar) la clase Tablero usada para representar los diferentes estados del juego y las interacciones entre ellos.

5.1 Representación de los jugadores (Clase AgenteEstudiante)

La clase AgenteEstudiante se utiliza para representar un jugador (es la clase equivalente a los agentes de las clases en ComportamientoJugador en la práctica anterior).

Contiene varios datos miembro:

Departamento de Ciencias de la Computación e Inteligencia Artificial

- **id** (int): Indica que jugador qué es (1 o 2)
- **profundidadMax** (int): Indica la profundidad máxima en la búsqueda
- **tiempoMaxSegundos** (double): Limite máximo de tiempo para realizar un movimiento
- **numHeuristica** (int): Selecciona la heurística a usar en el modo inteligente.
- **modo** (ModoJuego): Modo de operación del agente (ALEATORIO, MINIMAX, INTELIGENTE, STATUS)

Destacamos los siguientes métodos:

[IMPLEMENTADO]	std::pair<int, int> think (const Tablero& tablero)
	Esta es la función que invoca el simulador para procesar un movimiento del agente cuando no está en modo humano. Esta función se encarga de invocar, en función del modo de juego a JuegaAleatorio, Status o JuegaInteligente. Devuelve el movimiento en un pair fila, columna
[IMPLEMENTADO]	std::pair<int, int> JuegaAleatorio (const Tablero& tablero)
	Se invoca cuando está en modo de jugador aleatorio. Selecciona de forma aleatoria una de las posibles casillas que tiene libres en su turno. Devuelve el movimiento en un pair fila, columna
[Por IMPLEMENTAR]	Resultado Status (const Tablero &tablero, std::pair<int,int> &Mov)
	Cuando se pone en modo aplicar el método Status para resolver aplicando este algoritmo el tablero. Resultado es VICTORIA, EMPATE o DERROTA y en Mov se coloca el siguiente movimiento que lleva a obtener el Resultado.
[Por AMPLIAR]	std::pair<int, int> JuegaInteligente (const Tablero& tablero)
	Este es el método que se invoca cuando se elige el jugador inteligente. El estudiante debe terminar de hacer la implementación. En él se incluirá la llamada a la función <code>alfabeta</code>. Devuelve el movimiento en un pair fila, columna

[Por IMPLEMENTAR]	<code>double minimax(const Tablero &tablero, int profundidad, int prof_Max, std::pair<int,int> &Mov)</code>
	Aquí se debe implementar el método minimax. Los parámetros son el tablero de juego actual, la profundidad actual, la profundidad máxima y el movimiento seleccionado para jugar en el siguiente movimiento. Devuelve el valor minimax del árbol explorado como un valor real.
[Por IMPLEMENTAR]	<code>double alfaBeta(const Tablero &tablero, int profundidad, int prof_Max, double alfa, double beta, std::pair<int,int> &Mov)</code>
	Aquí se debe implementar el método alfabeta. Los parámetros son el tablero de juego actual, la profundidad actual, la profundidad máxima, los valores de alfa y beta y el movimiento seleccionado para jugar en el siguiente movimiento. Devuelve el valor minimax del árbol explorado como un valor real.
[Por AMPLIAR]	<code>double heuristica(const Tablero & tablero)</code>
	Este método parte de una implementación inicial. En él no está implementada directamente la heurística, sino que es una función que dentro de él tiene llamadas a todas las heurísticas que el estudiante quiera implementar. <pre>double AgenteEstudiante::heuristica(const Tablero & tablero) { switch(numHeuristica) { case 0: return heuristicaPrueba(tablero); break; case 1: return heuristica1(tablero); break; case 2: return heuristica2(tablero); break; default: return heuristica1(tablero); } }</pre>

	La idea es que el estudiante vaya añadiendo tantas heurísticas como desee y les vaya asociando un número. Hay dos números ya predefinidos, el 0 en el que obligatoriamente debe estar la llamada a la función <code>heuristicaPrueba</code> y el 1 que se reservará para la heurística que el estudiante considere mejor para competir contra los ninjas. La forma de invocar a cada heurística para el juego será en la llamada al simulador incluir <code>-id1 <num></code> o <code>-id2 <num></code>, siendo <code>num</code> el número asociado a la heurística deseada.
[IMPLEMENTADO]	<code>double heuristicaPrueba (const Tablero& tablero)</code>
	La implementación de una heurística que el estudiante no puede modificar ni cambiar en nada. Se usará para testear el comportamiento de los métodos <code>minimax</code> y <code>alfabeta</code> . Siempre estará situada con el número 0 en el case de la función anterior.
[Por IMPLEMENTAR]	<code>double heuristica1 (const Tablero& tablero)</code>
	El estudiante puede implementar tantos métodos <code>heuristica<num></code> que desee. En ellos es donde realmente estará la inteligencia del jugador del 5 en raya. Recuerda añadir cada vez que implementes una al método <code>heuristica</code> . Recuerda también que no podrá ser nunca la posición 0 y que la posición 1 estará destinada para tu mejor heurística. La que se usará para jugar contra los ninjas en la competición.
[IMPLEMENTADO]	<code>std::pair<int, int> SacarMovimiento(const Tablero& padre, const Tablero &hijo)</code>
	Método que devuelve el movimiento realizado por el jugador que le tocaba mover en el tablero padre y que da como resultado el tablero hijo. Es un método básico para la implementación de <code>minimax</code> y <code>alfabeta</code> y poder sacar la jugada realizada una vez seleccionado el mejor tablero.

5.2 La clase Tablero

La clase tablero será muy útil para construir las distintas heurísticas que vayáis realizando. Existen métodos en esta clase que permiten sacar todos los datos necesarios para poder hacer una buena evaluación del tablero. Podéis mirar directamente en los archivos `tablero.ccp/hpp` (Recordad que no se puede modificar nada en ellos) para ver la forma concreta en la que se implementa cada cosa. Aquí os ponemos una breve descripción de sus métodos.

La clase Tablero es el núcleo del motor de juego “Ninja 9x9”. Gestiona el estado de la partida, las reglas de la Trinidad (Módulo 3) y los efectos de las casillas especiales.

Métodos de Control de Turnos

`int getJugadorTurno() const`

Devuelve el identificador del jugador (1 o 2) al que le toca realizar el próximo movimiento según el reloj global.

`int getFaseActual() const`

Devuelve el número de fase de la Trinidad actual. Equivale al número total de turnos de jugador que se han completado (empieza en 2). La fase solo avanza cuando un jugador agota todas sus piezas y cede el turno al rival.

`int getTurnoActual() const`

Devuelve el número total de *piezas colocadas* en el tablero (también conocido como plies).
* **Nota:** Empieza en 4, asumiendo que ya se ha colocado el “Diamante Ninja” central inicial. Este contador aumenta con cada pieza puesta exitosamente, e incluye las piezas colocadas como resultado del bono verde.

``std::vector<Tablero> getSucesores() const``

Genera y devuelve todos los estados de tablero posibles a los que se puede llegar desde el estado actual con un único movimiento legal. Es el método fundamental para alimentar los algoritmos de búsqueda (**Status/MiniMax/Alfa-Beta**).

* **Nota:** Aplica internamente todas las reglas (Trinidad, Adyacencia y Efectos) y solo devuelve tableros resultantes de movimientos válidos.

std::vector<std::pair<Tablero, std::pair<int,int>>> getSucesoresConMovimientos() const

Variante de **getSucesores** que devuelve, además del tablero resultante, las coordenadas (fila, columna) del movimiento realizado para llegar a él. Muy útil para recuperar la mejor jugada en la raíz del árbol de búsqueda.

int getMovimientosRestantes() const

Devuelve el número de piezas que el jugador actual aún tiene derecho a colocar antes de que su turno termine y pase al oponente (parte de la secuencia 1-2-2-2).

bool ponerPieza(int fila, int columna, int jugador)

Intenta colocar una pieza. Aplica automáticamente las reglas de adyacencia y el ciclo de Módulo 3. Procesa también los efectos de casillas rojas, verdes y amarillas.

* **Retorno:** true si el movimiento es legal, false en caso contrario.

void pasarTurno()

Avanza el cronograma global del juego sin colocar pieza. Se usa si un jugador no tiene movimientos válidos.

Métodos de Consulta y Evaluación

int getCelda(int fila, int columna) const

Devuelve el contenido de una celda: 0 (vacío), 1 (Jugador 1) o 2 (Jugador 2).

TipoCelda getTipoCelda(int fila, int columna) const

Devuelve la especialidad de la casilla: NORMAL, ROJO, VERDE o AMARILLO.

int comprobarGanador()

Escanea el tablero en busca de una línea de longitud n.

* **Retorno:** ID del ganador, 0 si no hay, o -1 si hay empate técnico.

int getGanadorDesempate() const

Calcula el ganador por puntos cuando el tablero está lleno, contando líneas de piezas.

Departamento de Ciencias de la
Computación e Inteligencia Artificial

int contarCombinaciones(int n, int jugador) const

Cuenta cuántas líneas de longitud exacta n tiene el jugador especificado. Fundamental para la implementación de heurísticas.

bool tieneMovimientosValidos() const

Verifica si el jugador actual tiene alguna casilla legal disponible.

6. Evaluación y entrega de la práctica

La calificación final de la práctica se calculará de la siguiente forma:

- Se entregará una memoria de prácticas al finalizar las tareas a realizar. Es importante que en la memoria se refleje todas las implementaciones realizadas. Las obligatorias **Status**, **minimax** y **alfabeta**, más aquellas variantes que el estudiante quiera introducir (previa consulta a su profesor) por deseo propio. También debe reflejar las heurísticas implementadas, así como los éxitos o fracasos obtenidos (y los motivos por los que se piensa que se han obtenido).

La memoria contendrá una tabla resumen de resultados de cada una de las versiones de la búsqueda y de la heurística contra los ninjas disponibles. **Queda terminantemente prohibido introducir fragmentos de código en la memoria.**

- La práctica se califica numéricamente de 0 a 10. Se evaluará como la suma de los siguientes criterios:
 - La memoria de prácticas se evalúa de 0 a 2. Se evaluará la claridad de las explicaciones así como la originalidad de las heurísticas contempladas.
 - La correcta implementación de los métodos Status, minimax y alfabeta se evalúa de 0 a 2.
 - La eficacia del algoritmo se evaluará de 0 a 6 puntos y estará basado en competir frente a cuatro jugadores ninja tanto de jugador 1 como de jugador 2. Con estas 8 partidas (4 como primer jugador y 4 como segundo jugador) se definirá la capacidad de la heurística desarrollada por el estudiante. La calificación será de 1 punto por cada victoria y de 0.5 puntos por cada empate.

Para la competición la máxima profundidad a la que se puede aplicar el **minimax** será **profundidad 4** y para la **poda alfabeta** será **profundidad 7**. Recordad que, en los códigos de las heurísticas, **heuristicaPrueba** debe estar siempre asociada al **índice 0** y la **heurística que proponéis para la competición con el índice 1**. Para estas versiones, el parámetro tiempo (-t) no será de aplicación.

Para competir contra los ninjas será necesario levantar la [VPN de la UGR](#). Si no es así, el software dará error indicando que no están los ninjas accesibles.

La fórmula que se aplicará para obtener los puntos del estudiante en la competición será

$$\min(\text{Puntos Obtenidos}, 6)$$

- La nota final será la suma de los apartados anteriores. Es requisito necesario entregar tanto el software como la memoria de prácticas. Si alguna de ellas falta en la entrega se entenderá por no entregada.
- La fecha de entrega de la práctica **Domingo 31 de Mayo antes de las 23:00 horas**.
 - La entrega consistirá en un fichero de nombre practica3.zip que contendrá 3 archivos: los archivos **AgenteEstudiante.cpp**, **AgenteEstudiante.hpp** que definen el comportamiento de nuestro jugador inteligente de 5 en raya, y **Memoria.pdf** con la memoria de las prácticas.
 - **La falta de alguno de estos tres archivos en la entrega final supondrá a efectos prácticos no haber entregado las prácticas obteniéndose una calificación de 0**. Por tanto, se recomienda que, tras la subida de la práctica, el estudiante se asegure que están esos 3 archivos y sólo esos tres.
- La **memoria de prácticas** debe ser un documento de pocas páginas (¡sin código!) en formato PDF que, como mínimo, contenga los siguientes apartados:
 - Listado y breve descripción de las heurísticas propuestas.
 - Tabla comparativa de todas (al menos de las más relevantes) las heurísticas propuestas con los ninjas y entre ellas. En la tabla se identificará el número asignado por el estudiante a la heurística para invocarse por su software.
 - Análisis de los resultados obtenidos.
 - Opcionalmente, se puede incluir una reflexión personal sobre la práctica.

IMPORTANTE: Para la competición contra los ninjas se corregirá con -id1 y -id2 con valor 1, así que asociad la mejor heurística a la posición 1. La versión de la poda asociada al 0 debe ser obligatoriamente heurísticaPrueba.

7. Observaciones Finales

Esta **práctica es INDIVIDUAL**. El profesorado para asegurar la originalidad de cada una de las entregas someterá a estas a un procedimiento de detección de copias. En el caso de detectar prácticas copiadas, los involucrados (tanto el que se copió como el que se ha dejado copiar) tendrán suspensa la asignatura. Por esta razón, recomendamos que en ningún caso se intercambie código entre los estudiantes. No servirá como justificación del parecido entre dos prácticas el argumento *“es que la hemos hecho juntos y por eso son tan parecidas”*, o *“como estudiamos juntos, pues...”*, ya que como se ha dicho antes, **las prácticas son INDIVIDUALES**.

Como se ha comentado previamente, el objetivo de la defensa de prácticas es evaluar la capacidad del estudiante para enfrentarse a este problema. Por consiguiente, se asume que todo el código que aparece en su práctica ha sido introducido por el estudiante por alguna razón y que, por lo tanto, domina perfectamente el código que entrega. Así, si durante cualquier momento del proceso de defensa el estudiante no justifica adecuadamente algo de lo que aparece en su código, la práctica se considerará copiada y, por tanto, suspensa, reservándose los profesores elevar a instancias superiores si la falta se considerará grave. Por esta razón, aconsejamos que el estudiante no incluya nada en su código que no sea capaz de explicar qué misión cumple dentro de su práctica y que revise el código con anterioridad a la defensa de prácticas.

Por último, las prácticas presentadas en tiempo y forma, pero que no vengan acompañadas de la entrega del documento de autoevaluación realizado por el estudiante, se considerarán como no entregadas y por consiguiente se obtendrá la calificación de 0. El supuesto anterior se aplica a aquellas prácticas no involucradas en un proceso de copia. En este último caso, el estudiante podría tener una penalización más grave.